


```
int sod(N) {
```

// Ass:- Given N, Calculate & return sum of digits.

```
// Base: if (N == 0) {  
    return 0  
}
```

// Main logic: $N = 34678$

$$N \% 10 = 8$$

$$N / 10 = 3467$$

$$\rightarrow 8 + \text{sod}(3467)$$

```
    return N % 10 + sod(N / 10);
```

```
}
```

$$123 \% 10 = 3$$

$$\hookrightarrow 123 / 10 = 12 \% 10 = 2$$

$$\hookrightarrow 12 / 10 = 1$$

$$1 \% 10 = 1$$

$$\hookrightarrow 1 / 10 = \underline{0}$$

$\text{pow}(a, N)$ - Calculate & return a^N

Note: No usage of inbuilt functions
Implement using Recursion.

1st approach:-

$$a^5 = a * a^4$$

$$a^{10} = a * a^9$$

$$\boxed{a^N = a * a^{N-1}}$$

$\downarrow$
 $\text{pow}(a, N-1)$

$a^0 = 1$

```
pow(a, n) {  
    if (n == 0) return 1;  
    return a * pow(a, n-1);  
}  
  
if (n == 1) { return a }
```

2nd approach:-

$$a^{10} = a^5 * a^5 \quad 10/2 = 5$$

$$a^{12} = a^6 * a^6 \quad 12/2 = 6$$

$$a^{13} = a^6 * a^6 * a \quad 13/2 = 6$$

$$a^{15} = a^7 * a^7 * a \quad 15/2 = 7$$

$$a^{18} = a^9 * a^9 \quad 18/2 = 9$$

Idea:-

a^N :

if ($N \cdot 1/2 == 0$) {

$$a^{N/2} * a^{N/2}$$

}

else {

$$a^{N/2} * a^{N/2} * a$$

}

Pseudo Code :-

$\text{pow}(a, N) \{$

if ($N == 0$) return 1

he = $\text{pow}(a, N/2)$

ha = $\text{he} * \text{he}$

if ($N \% 2 == 0$) {

return ha

}

else {

return ha * a

}

}

$TC - O(\log_2 N)$

{Fast Exponentiation}

$\hookrightarrow \log_2 N$

$\text{pow}(a, 10) \{$

he = $\text{pow}(a, 5)$

ha = $\text{he} * \text{he}$

= $a^5 * a^5$

= a^{10}

return a^{10}

}

$\text{pow}(a, 5) \{$

he = $\text{pow}(a, 2)$

ha = $\text{he} * \text{he}$

= $a^2 * a^2$

= a^4

return $a^4 * a$

}

$\text{pow}(a, 2) \{$

he = $\text{pow}(a, 1)$

ha = $\text{he} * \text{he} = a^1$

return ha

a^2

}

$\text{pow}(a, 1) \{$

he = $\text{pow}(a, 0)$

ha = 1

return ha * a

{

$1 * a$

}

Time function of $N = T(N)$

```

int Sum(N) {
    Ass:- Given N, calculate sum of [1 N] & return it
    if (N==1) return 1
    return Sum(N-1) + N } Recursive equations

```

Substitution
Method.

$$T(N) = O(1) + T(N-1) \quad T(1) = 1$$

$$T(N) = T(N-1) + 1$$

$$T(N-1) = T(N-2) + 1$$

$$T(N) = T(N-2) + 2 \Rightarrow \underbrace{T(N-3) + 1}_{T(N-2)} + 2 = T(N-3) + 3$$

$$T(N-2) = T(N-3) + 1$$

$$T(N) = T(N-3) + 3$$

$$T(N-3) = T(N-4) + 1$$

$$T(N) = T(N-4) + 4$$

$$T(1) = 1$$

$$N - K = 1$$

$$K = N - 1$$

$$TC: O(N)$$

Generalized equation:-

$$T(N) = T(N-K) + K$$

$$T(N) = 1 + N - 1 = N$$

int fact(N) $\rightarrow T(N)$

{ if (N == 1) { return 1; }

{ return N * fact(N-1) $\rightarrow T(N-1)$ }

$$T(N) = O(1) + T(N-1) + O(1)$$

$$T(N) = O(1) + T(N-1)$$

$$T(N) = O(N) \checkmark$$

```

pow(a, N) {
  if (N == 0) return 1
  he = pow(a, N/2)
  ha = he * he
  if (N % 2 == 0) {
    return ha
  }
  else {
    return ha * a
  }
}

```

$$T(N) = O(1) + T(N/2)$$

$$T(1) = 1$$

$$T(0) = 1$$

```

pow(a, 1) {
  he = pow(a, 0) — O(1)
  ha = he * he — O(1)
  return ha * a — O(1)
}

```

$$T(N) = T(N/2) + 1$$

$$\uparrow T(N/2) = T(N/4) + 1$$

$$T(N) = T(N/4) + 2 \quad \} \quad T(N/2^2) + 2$$

$$\uparrow T(N/4) = T(N/8) + 1$$

$$T(N) = T(N/8) + 3 \quad \} \quad T(N/2^3) + 3$$

$$\uparrow T(N/8) = T(N/16) + 1$$

$$T(N) = T(N/16) + 4 \quad \} \quad T(N/2^4) + 4$$

Generalized equation :-

$$T(N) = T(N/2^k) + k$$

$$T(N) = 1 + \log_2 N$$

$$TC: O(\log_2 N)$$

$$T(1) = 1$$

$$\frac{N}{2^k} = 1$$

$$2^k = N$$

Take $\log_2$ both sides

$$\log_2 2^k = \log_2 N$$

$$k = \log_2 N$$

$$T(N) = 2T(N/2) + 1$$

$$T(1) = 1$$

$$T(N/2) = 2T(N/4) + 1$$

$$T(N) = 2[2T(N/4) + 1] + 1$$

$$T(N) = 4T(N/4) + 3$$

$$T(N) = 2^2 T\left(\frac{N}{2^2}\right) + (2^2 - 1)$$

$$T(N/4) = 2T(N/8) + 1$$

$$T(N) = 4[2T(N/8) + 1] + 3$$

$$T(N) = 8T(N/8) + 7$$

$$T(N) = 2^3 T\left(\frac{N}{2^3}\right) + (2^3 - 1)$$

$$T(N/8) = 2T(N/16) + 1$$

$$T(N) = 8[2T(N/16) + 1] + 7$$

$$T(N) = 16T(N/16) + 15$$

$$T(N) = 2^4 T\left(\frac{N}{2^4}\right) + (2^4 - 1)$$

Generalized eq.

$$T(N) = 2^k T\left(\frac{N}{2^k}\right) + (2^k - 1)$$

$$T(1) = 1$$

$$\frac{N}{2^k} = 1 \Rightarrow k = \log_2 N$$

$$T(N) = 2^{\log_2 N} T(1) + 2^{\log_2 N} - 1$$

$$= N + N - 1$$

$$TC :- O(N)$$

$$2^{\log_2 N} = N$$

int fib(N) {

if (N==1 or N==2) return 1 } O(1)
return fib(N-1) + fib(N-2) }
}

$$T(N) = T(N-1) + T(N-2) + 1$$

$$\begin{cases} T(N-1) = T(N-2) + T(N-3) + 1 \\ T(N-2) = T(N-3) + T(N-4) + 1 \end{cases}$$

$$T(N) = T(N-2) + T(N-3) + T(N-3) + T(N-4) + 3$$

$$T(N-2) + 2T(N-3) + T(N-4) + 3$$

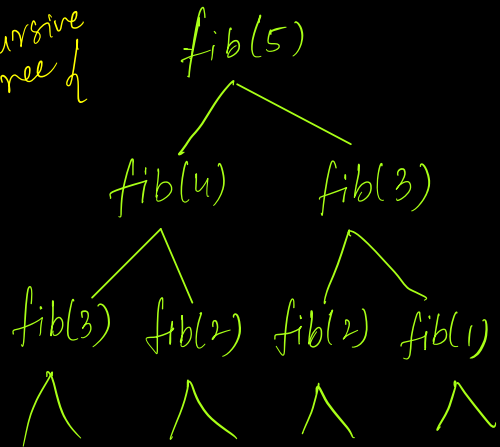
{ If more than 1 time term is present on right }
better to go with Recursive tree method }

```

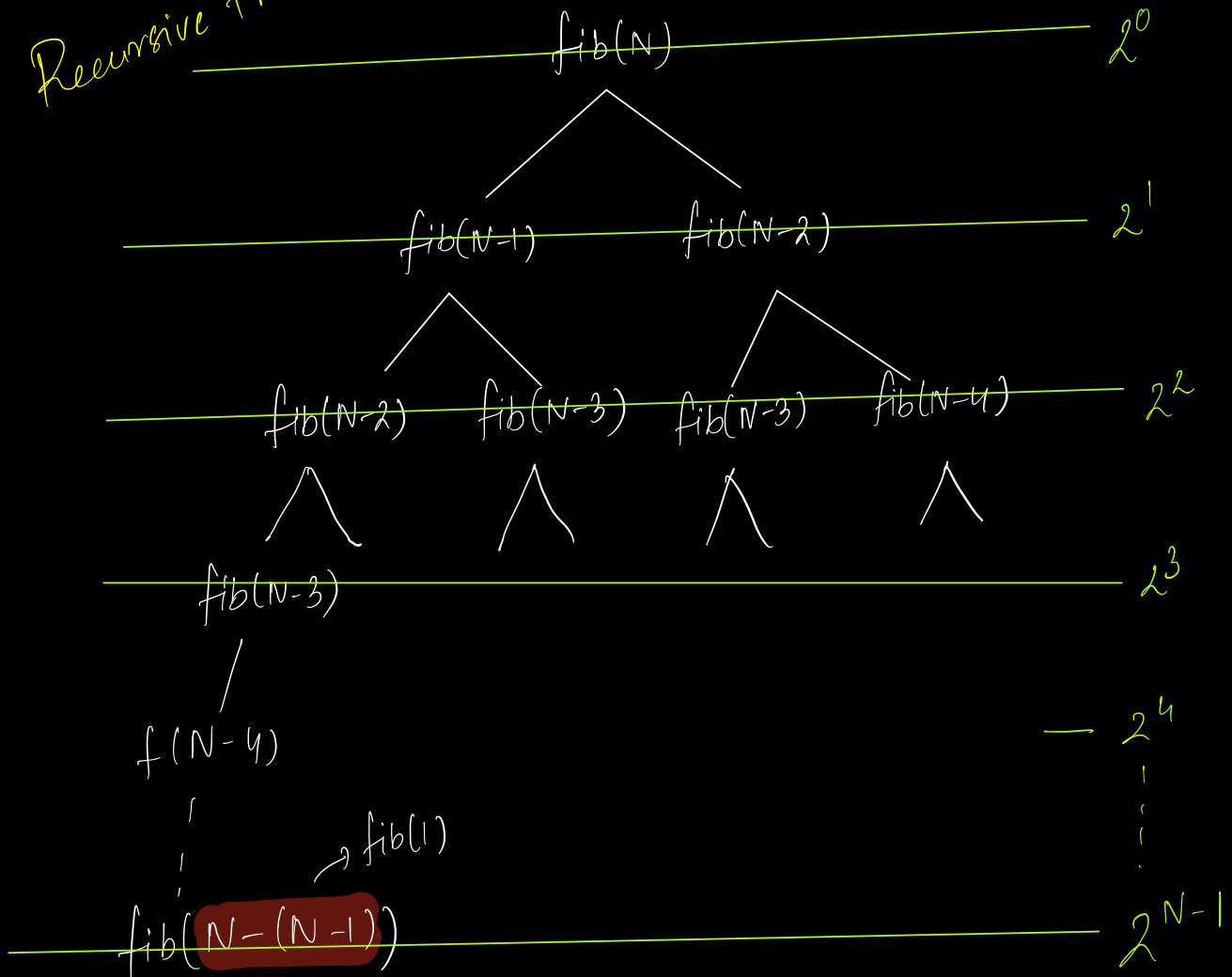
int fib(N) {
    if (N == 1 or N == 2) return 1
    return fib(N-1) + fib(N-2)
}

```

Recursive tree



Recursive tree Method:-



$$\text{Total} = 2^0 + 2^1 + 2^2 + \dots + 2^{N-1} \} \text{ GP}$$

$$a = 1 \quad \text{Sum} = \frac{a \times (r^N - 1)}{(r - 1)} = \frac{1 \times (2^N - 1)}{(2 - 1)}$$

$$r = 2$$

$$N = N$$

$$= 2^N - 1$$

$$TC: O(2^N)$$

Space Complexity :- Max stack size at a given point.

```
int Sum(N) {
```

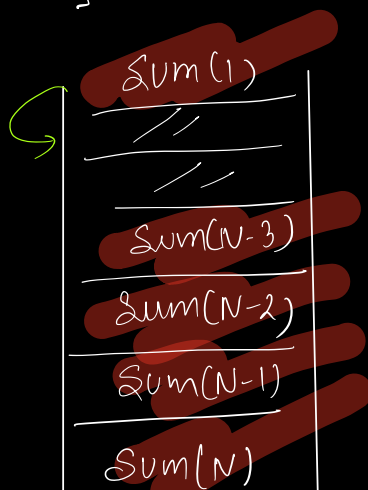
Ass:- Given N, Calculate Sum of [1 N] & return it

```
if (N == 1) return 1
```

```
return Sum(N-1) + N
```

```
}
```

SC: $O(N)$



$Sum(N)$

$\hookrightarrow Sum(N-1)$

$\hookrightarrow Sum(N-2)$

$\hookrightarrow Sum(N-3)$

$\hookrightarrow$

$Sum(1)$

```
int fib(N) {
```

```
    if (N == 1 or N == 2) return 1
```

```
    return fib(N-1) + fib(N-2)
```

```
}
```

4/5

TC: $O(2^N)$

SC: $O(N)$

